

ĐỀ SỐ 2

XÂY DỰNG ỨNG DỤNG ASP.NET CORE WEB API CÓ TÊN: <HọTên> <MãSốSinhViên> (ví dụ: LeVanMinh20241123)

🕒 Thời gian làm bài: 120 phút

1. Yêu cầu chung

a. Tổ chức code

- Tuân theo coding convention:
 - PascalCase cho tên các class, interface, method.
 - camelCase cho biến thông thường.
 - _camelCase cho các biến private.
- Cấu trúc thư mục:

```
- Properties
- Constants
- Controllers
- DbContexts
- Dtos
  - Mechanic
  - Vehicle
- Entities
- Exceptions
- Migrations
- Services
  - Implements
  - Interfaces
- Utils
- appsettings.json
- Program.cs
```

b. Quy định triển khai từng thành phần

- **DTOs:**
 - Các class Dto phải xử lý `trim()` cho các trường string.
 - Validate đầu vào sử dụng các thuộc tính built-in như `[Required]`, `[StringLength]`, `[Range]`,...
- **Entity:**
 - Sử dụng kiểu dữ liệu phù hợp với bài toán thực tế.
 - Bắt buộc có khóa chính (PK) là số nguyên tự tăng.

- Quan hệ khóa ngoại được cấu hình đầy đủ.
 - **Controllers:**
 - Trả về kiểu `ActionResult` và sử dụng `Ok()`, `BadRequest()`, `NotFound()`,...
 - **Exception:**
 - Sử dụng lớp `UserFriendlyException` để trả về thông báo có thể hiểu được đối với người dùng.
 - **EF Core & DB:**
 - Sử dụng Code First để tạo migration và cập nhật vào SQL Server.
 - Các truy vấn sử dụng Linq method hoặc Linq query syntax.
 - **Service & DI:**
 - Logic được đóng gói vào lớp service và inject vào controller.
 - Khuyến khích sử dụng interface.
 - **Kế thừa:**
 - Cho phép kế thừa các class để tái sử dụng (tùy chọn).
 - **Đặt tên class:**
 - Theo định dạng:

```
<TênClass><MãSốSinhViên>De3<GiờHiệnTại>
```

 - `<GiờHiệnTại>` là giờ phút hiện tại trên máy khi làm bài (ví dụ: 1035 cho 10:35 sáng).
 - Ví dụ: `MechanicDto20241123De31035`, `RepairRecordController20241123De31035`
 - Các file tự sinh từ migration (trong thư mục `Migrations/`) không cần đổi tên.
-

2. Bài toán cụ thể

Thiết kế hệ thống theo dõi hoạt động sửa chữa phương tiện giao thông tại một trung tâm bảo dưỡng.

Các bảng và quan hệ:

- **Mechanic**
 - Id (int, PK, tự tăng)
 - MaTho (string, unique, bắt buộc)
 - TenTho (string, unique, bắt buộc)
 - CCCD (string, bắt buộc, unique)
 - NgayNhanViec (DateTime)

- **Vehicle**
 - Id (int, PK, tự tăng)
 - BienSoXe (string, unique, bắt buộc)
 - LoaiXe (string, bắt buộc)
 - **RepairRecord** (quan hệ nhiều-nhiều giữa Mechanic và Vehicle)
 - Id (int, PK, tự tăng)
 - MechanicId (FK)
 - VehicleId (FK)
 - SoLanSua (int, ≥ 0)
-

3. Yêu cầu thực hiện

a. Cơ sở dữ liệu

- Tạo migration và cập nhật vào cơ sở dữ liệu toàn bộ các bảng và quan hệ trên.
- Đảm bảo tạo ràng buộc khoá chính, khoá ngoại.

b. API xử lý nghiệp vụ

1. API thêm, sửa, xóa thợ sửa chữa (mechanic)

- Kiểm tra trùng mã, tên, CCCD khi thêm và sửa.
- Sử dụng `UserFriendlyException` để xử lý lỗi nghiệp vụ.

2. API xem danh sách thợ sửa chữa

- Kèm phân trang: `PageIndex`, `PageSize`
- Có hỗ trợ lọc gần đúng theo `TenTho` hoặc `CCCD` (gợi ý dùng trường `Keyword` trong DTO filter).
- Kết quả trả về dạng: tổng số trang, danh sách thợ ở trang hiện tại.

3. API liệt kê danh sách xe được sửa nhiều nhất bởi một thợ

- Đầu vào: `MechanicId`
 - Đầu ra: danh sách gồm `LoaiXe`, `BienSoXe`, sắp xếp giảm dần theo `BienSoXe`.
 - Chỉ liệt kê các xe có số lần sửa chữa lớn nhất do người thợ đó thực hiện.
 - Gợi ý: Tìm số lần lớn nhất, sau đó lọc theo `SoLanSua == Max`.
-

4. Yêu cầu xử lý ngoại lệ

- Tạo class `UserFriendlyException` kế thừa từ `Exception`, cho phép truyền thông điệp thân thiện đến người dùng thông qua constructor.
- Tại các controller, bao quanh lời gọi service bằng khối `try - catch`, nếu gặp `UserFriendlyException` thì trả về `BadRequest(...)` cùng thông báo lỗi.
- Ví dụ:

```
catch (UserFriendlyException ex)
{
    return BadRequest(new { message = ex.Message });
}
```

Ghi chú:

- Đọc kỹ đề bài trước khi bắt đầu làm bài.
- Cán bộ coi thi không giải thích gì thêm.
- Không sử dụng thư viện bên ngoài trừ khi được chỉ định.
- Sinh viên cần đặt tên class đúng quy ước, và đặt **namespace** rõ ràng theo từng phần.

----- **HẾT** -----